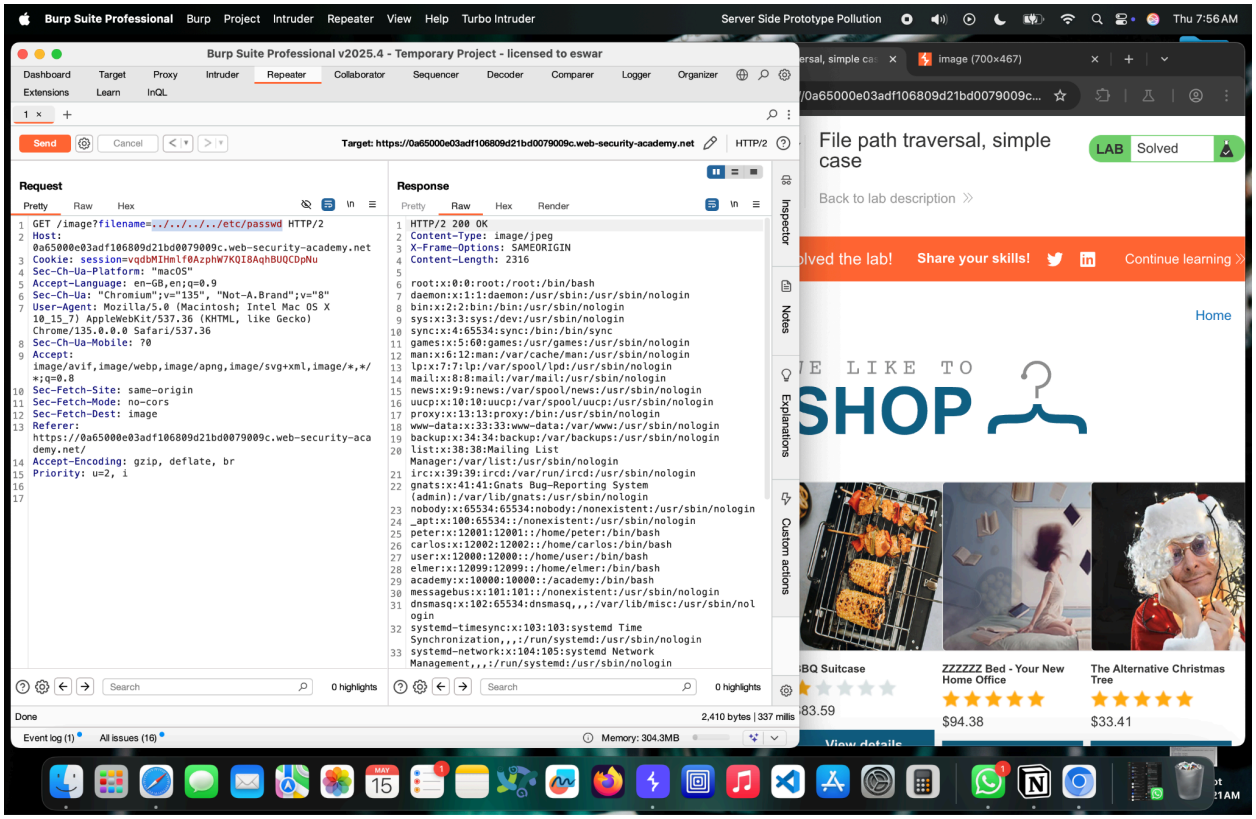
LAB 1: FILE PATH TRAVERSAL,SIMPLE CASE

Lab Name: File path traversal, simple case

Vulnerability Type: File Path Traversal



Summary: This lab demonstrates a basic file path traversal vulnerability. By manipulating the `filename` parameter in the URL, it's possible to access files outside the web root, such as `/etc/passwd`, leading to the exposure of sensitive system information.

Screenshot Context: The screenshot shows a Burp Suite Repeater request where the `filename` parameter is modified to include `..//etc/passwd`. The response on the right displays the contents of the `/etc/passwd` file.

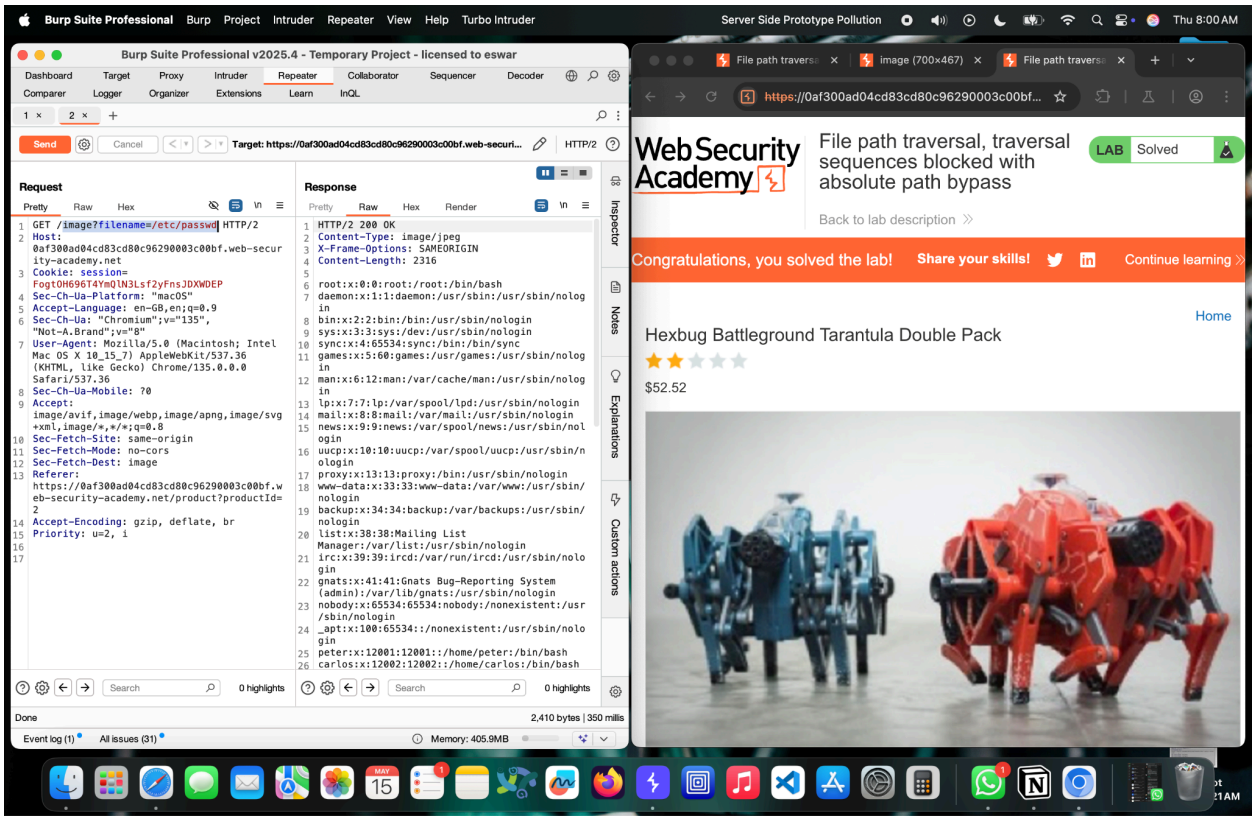
Key Takeaways:

- Improper input validation can lead to file path traversal.
- Accessing sensitive files like `/etc/passwd` can have severe security consequences.

LAB 2: FILE PATH TRAVERSAL,TRAVERSAL SEQUENCES BLOCKED WITH ABSOLUTE PATH BYPASS

Lab Name: File path traversal, traversal sequences blocked with absolute path bypass

Vulnerability Type: File Path Traversal / Absolute Path Bypass



Summary: This lab builds on the basic path traversal by implementing a defense that blocks `../` sequences. However, it demonstrates how this defense can be bypassed by using an absolute path (e.g., directly specifying `/etc/passwd`) to access the file.

Screenshot Context: The screenshot shows the web application interface indicating that traversal sequences are blocked. The solution likely involves crafting a request with an absolute file path to bypass this restriction.

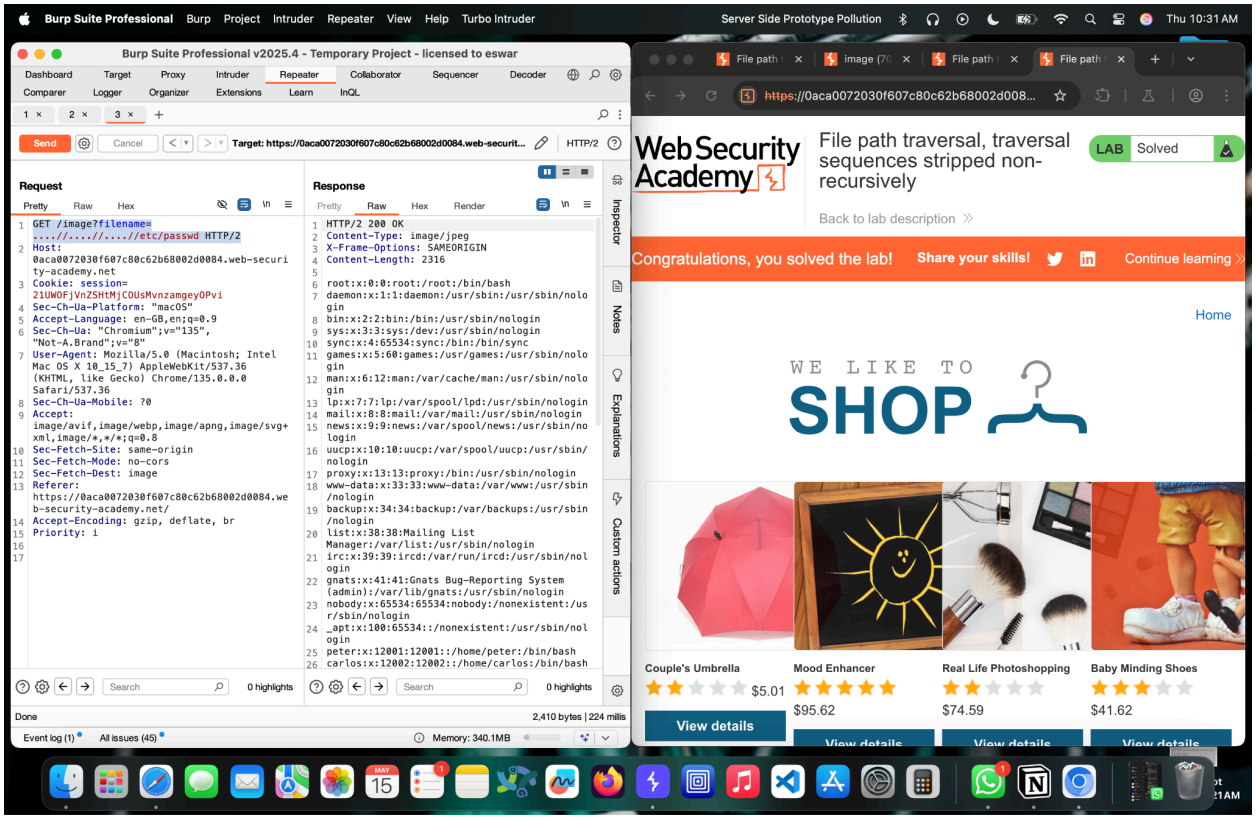
Key Takeaways:

- Simply blocking relative path traversal sequences is not always sufficient.
- Absolute path handling must also be secured.

LAB 3: FILE PATH TRAVERSAL,TRAVERSAL SEQUENCES STRIPPED NON-RECURSIVELY

Lab Name: File path traversal, traversal sequences stripped non-recursively

Vulnerability Type: File Path Traversal / Non-Recursive Stripping Bypass



Summary: This lab shows a scenario where the server attempts to sanitize user input by stripping `../` sequences. However, it demonstrates how this can be bypassed by using nested or repeated sequences (e.g., `....//`) because the stripping is non-recursive.

Screenshot Context: The screenshot shows the web application interface. The solution likely involves crafting a payload with nested or repeated traversal sequences.

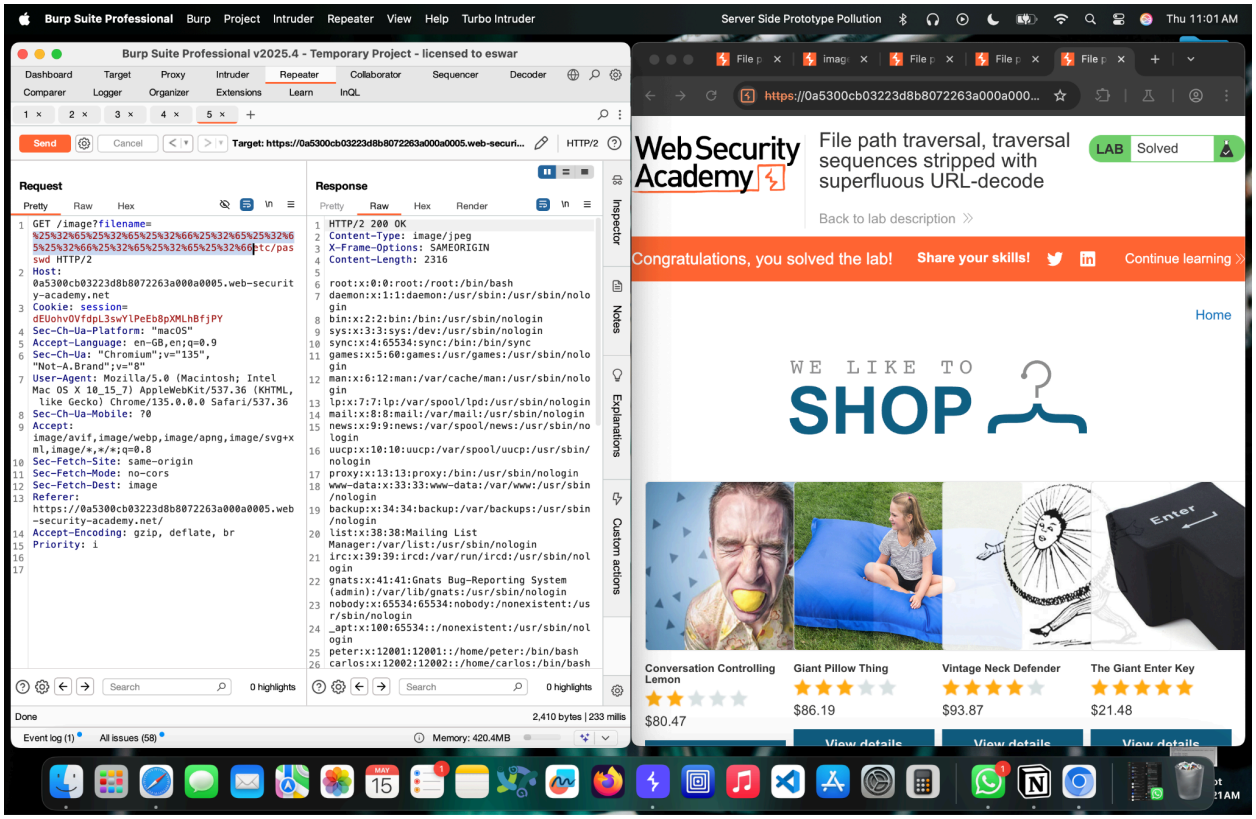
Key Takeaways:

- Input sanitization must be recursive to handle nested traversal sequences.
- Security measures should be applied iteratively until all potential bypasses are addressed.

LAB 4: FILE PATH TRAVERSAL,TRAVERSAL SEQUENCES STRIPPED WITH SUPERFLUOUS URL-DECODE

Lab Name: File path traversal, traversal sequences stripped with superfluous URL-decode

Vulnerability Type: File Path Traversal / URL-Decode Bypass



Summary: This lab introduces URL encoding as part of the defense mechanism. It demonstrates how to bypass this by providing URL-encoded traversal sequences that are decoded by the server *after* the stripping mechanism, effectively circumventing the defense.

Screenshot Context: The screenshot shows the web application interface. The solution likely involves encoding the traversal sequences in the request.

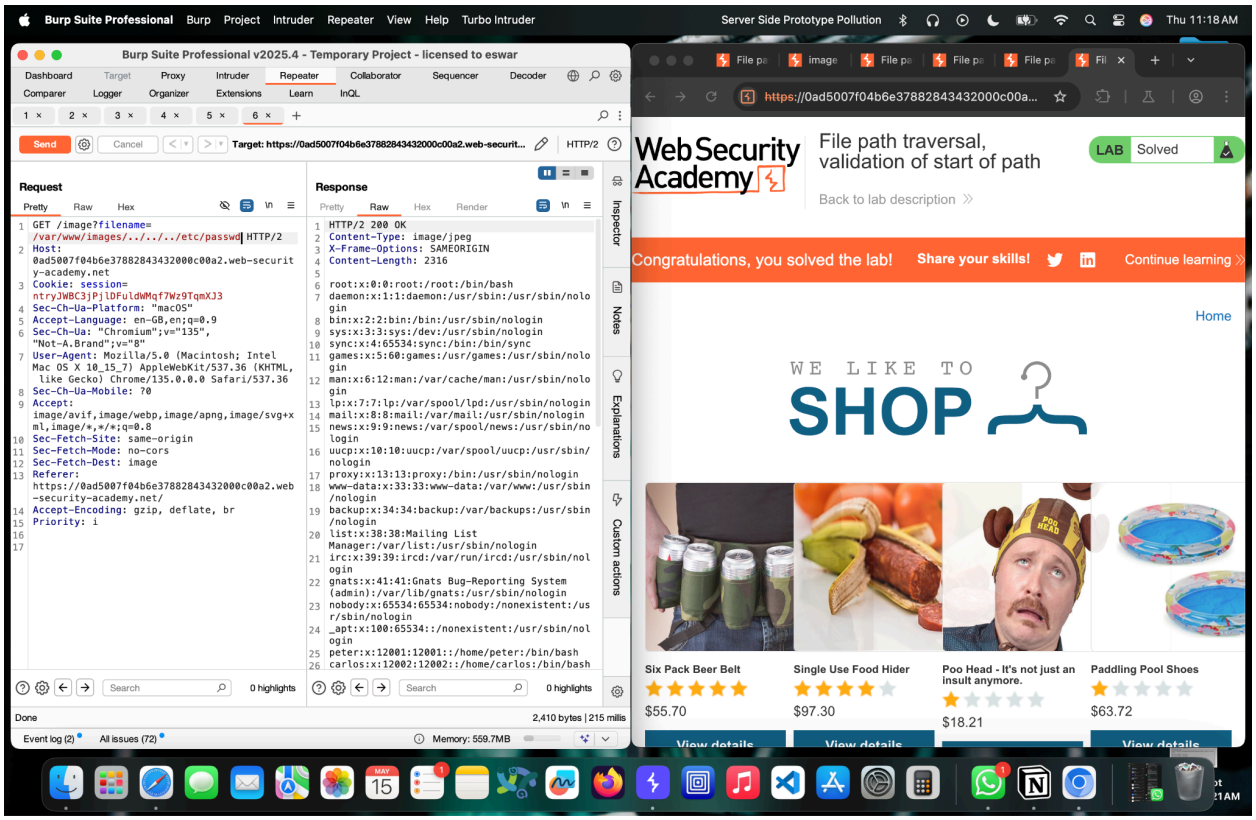
Key Takeaways:

- The order of decoding and filtering operations is critical in security.
- Decoding input before or after filtering can create vulnerabilities.

LAB 5: FILE PATH TRAVERSAL,VALIDATION OF START OF PATH

Lab Name: File path traversal, validation of start of path

Vulnerability Type: File Path Traversal / Path Prefix Bypass



Summary: This lab presents a scenario where the application validates that the requested path starts with an expected directory. It demonstrates how to bypass this by finding a way to include the target file (e.g., `/etc/passwd`) while still having the path start with the allowed prefix, or by exploiting a logical flaw in the validation logic.

Screenshot Context: The screenshot shows the web application interface. The solution likely involves manipulating the path to satisfy the prefix validation while still accessing the target file.

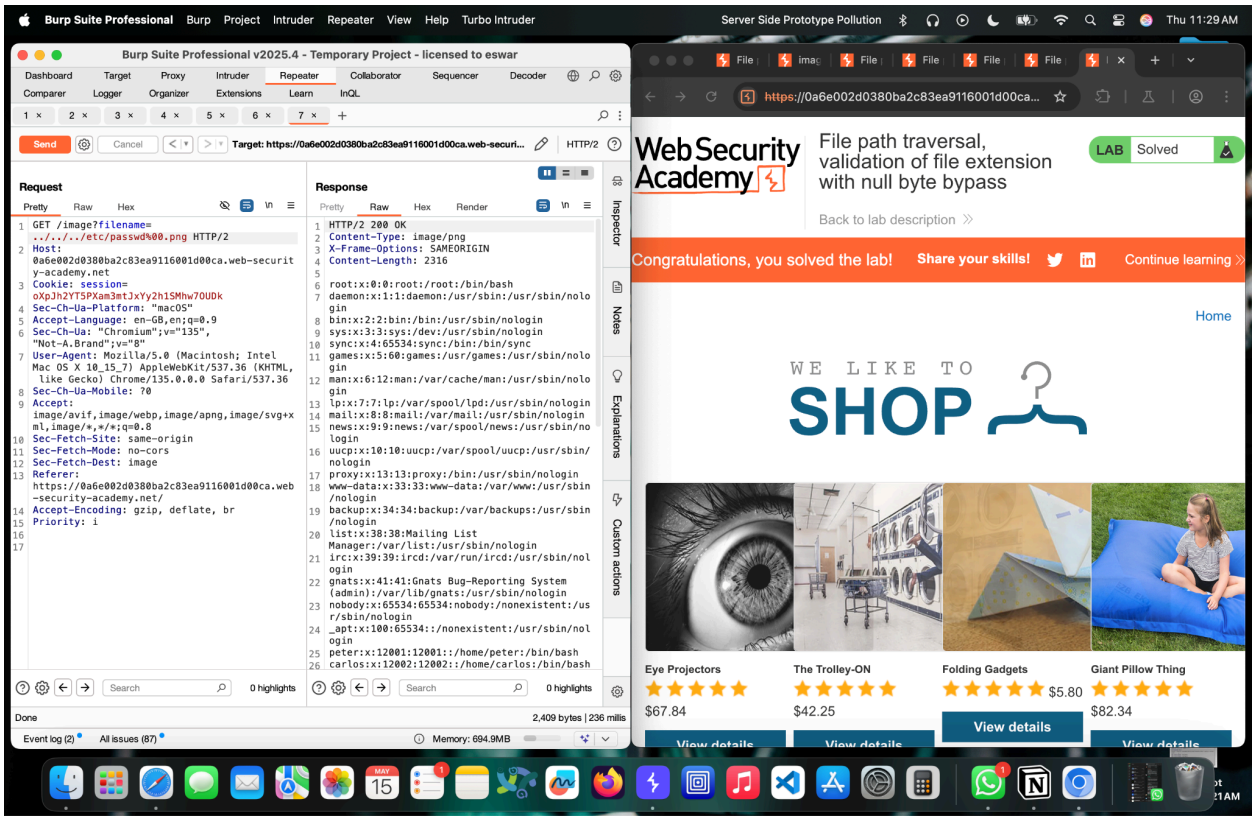
Key Takeaways:

- Path prefix validation can be bypassed if not implemented carefully.
- Logical flaws in validation routines can lead to vulnerabilities.

LAB 6: FILE PATH TRAVERSAL,VALIDATION OF FILE EXTENSION WITH NULL BYTE BYPASS

Lab Name: File path traversal, validation of file extension with null byte bypass

Vulnerability Type: File Path Traversal / Null Byte Injection



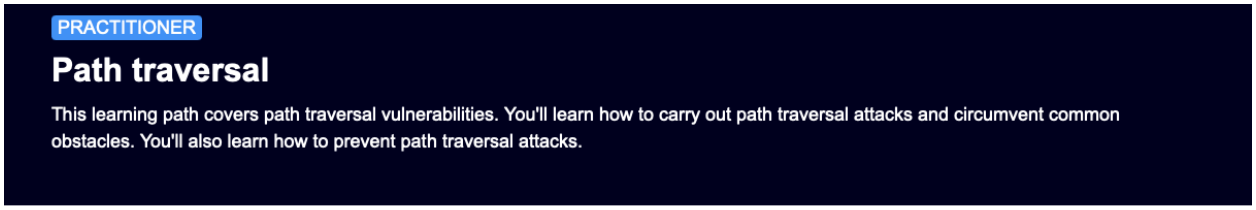
Summary: This lab involves a defense that checks if the requested file has an allowed extension. It demonstrates how to bypass this using a null byte (`%00`) in the filename, which can truncate the string and bypass the extension check in some systems.

Screenshot Context: The screenshot shows the web application interface. The solution likely involves injecting a null byte into the filename.

Key Takeaways:

- Null byte injection is a classic technique to bypass file extension checks.
- String handling vulnerabilities can have security implications.

Path traversal: Key Topics Explored



Contents

Restart: What is path traversal?

14 of 14

RESTART LEARNING

What is path traversal? 1 of 1

- What is path traversal? Restart

Reading arbitrary files via path traversal 2 of 2

- Reading arbitrary files via path traversal
- Lab: File path traversal, simple case APPRENTICE

Common obstacles to exploiting path traversal vulnerabilities 10 of 10

- Common obstacles to exploiting path traversal vulnerabilities
- Lab: File path traversal, traversal sequences blocked with absolute path bypass PRACTITIONER
- Common obstacles to exploiting path traversal vulnerabilities - Continued
- Lab: File path traversal, traversal sequences stripped non-recursively PRACTITIONER
- Common obstacles to exploiting path traversal vulnerabilities - Continued
- Lab: File path traversal, traversal sequences stripped with superfluous URL-decode PRACTITIONER
- Common obstacles to exploiting path traversal vulnerabilities - Continued
- Lab: File path traversal, validation of start of path PRACTITIONER
- Common obstacles to exploiting path traversal vulnerabilities - Continued
- Lab: File path traversal, validation of file extension with null byte bypass PRACTITIONER

How to prevent a path traversal attack 1 of 1

- How to prevent a path traversal attack